

main.ts

claude.md

From Developer to Director: Software Engineering with Generative AI Agents

A masterclass synthesis based on the Vanderbilt University curriculum by Dr. Jules White.

```
> claude orchestrate --scale 1000x  
[Success] Initializing AI development team...  
[Success] Ready for delegation.
```

The myth is better autocomplete. The reality is scalable AI labor.

Micromanaging

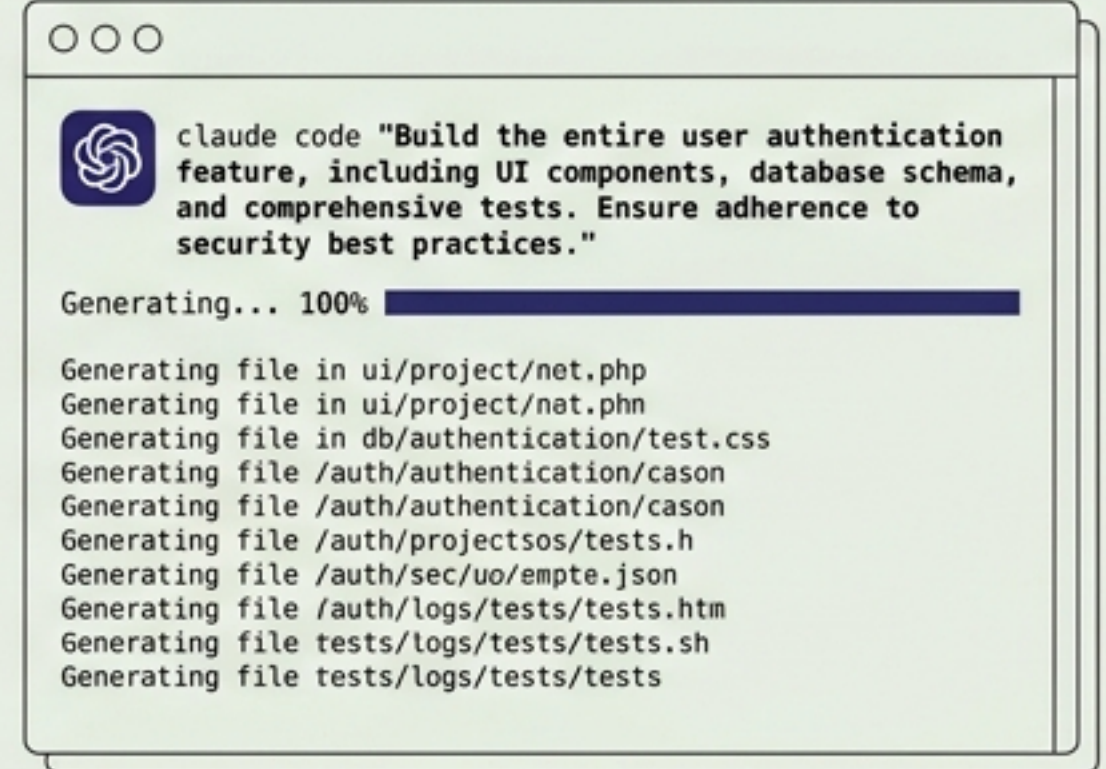
```
function calculateTotal() {  
  ... return a + b;  
}
```



Using AI to write single functions you copy-paste.

Orchestrating

```
├── ui  
├── db  
├── auth  
│   ├── add  
│   ├── adg  
│   ├── ecs  
│   ├── cdd  
│   ├── con  
│   ├── bin  
│   ├── bbi  
│   ├── bud  
│   ├── hoX  
│   ├── now  
│   ├── nso  
│   ├── ssp  
│   ├── uin  
│   └── vss  
└── tests
```

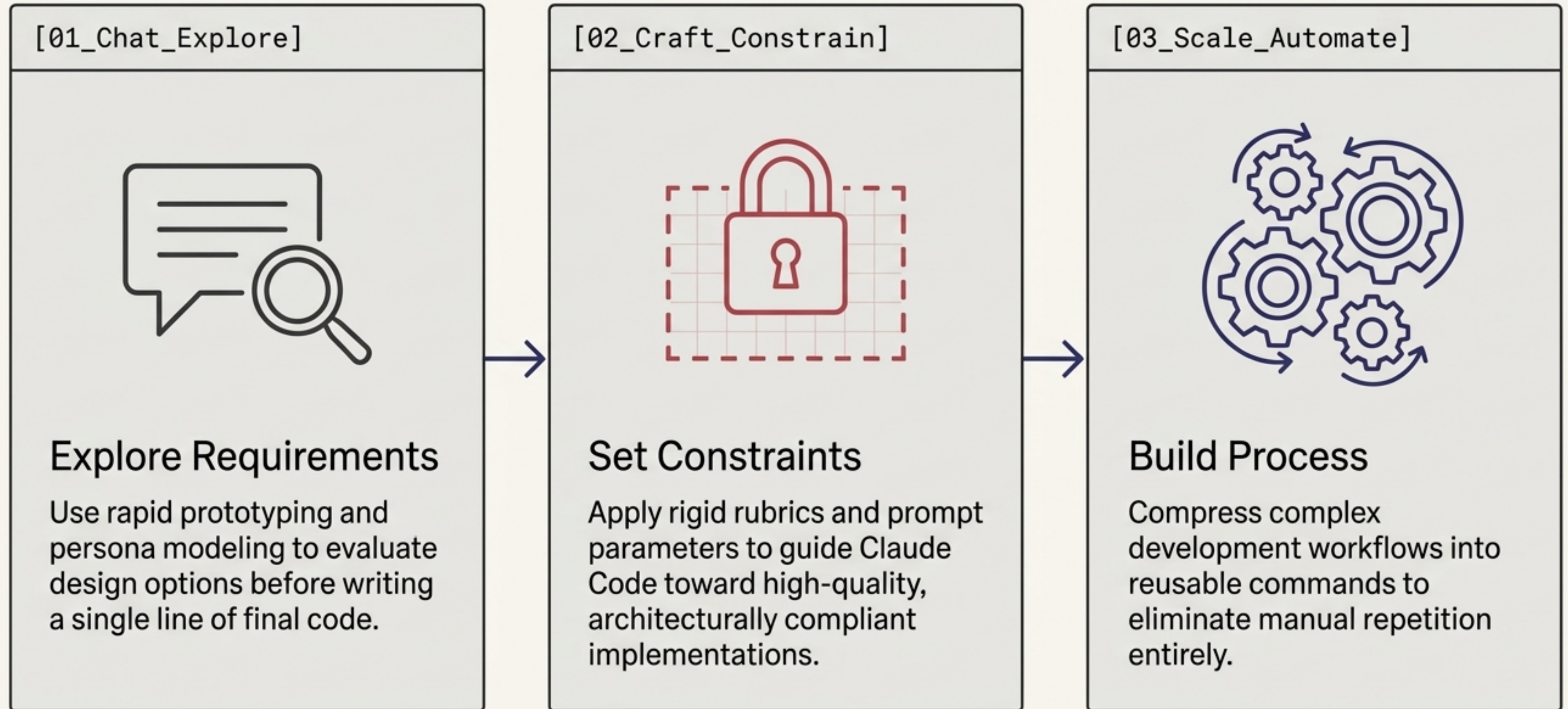


Orchestrating Claude Code like a tech lead managing a team of senior developers to build entire features.

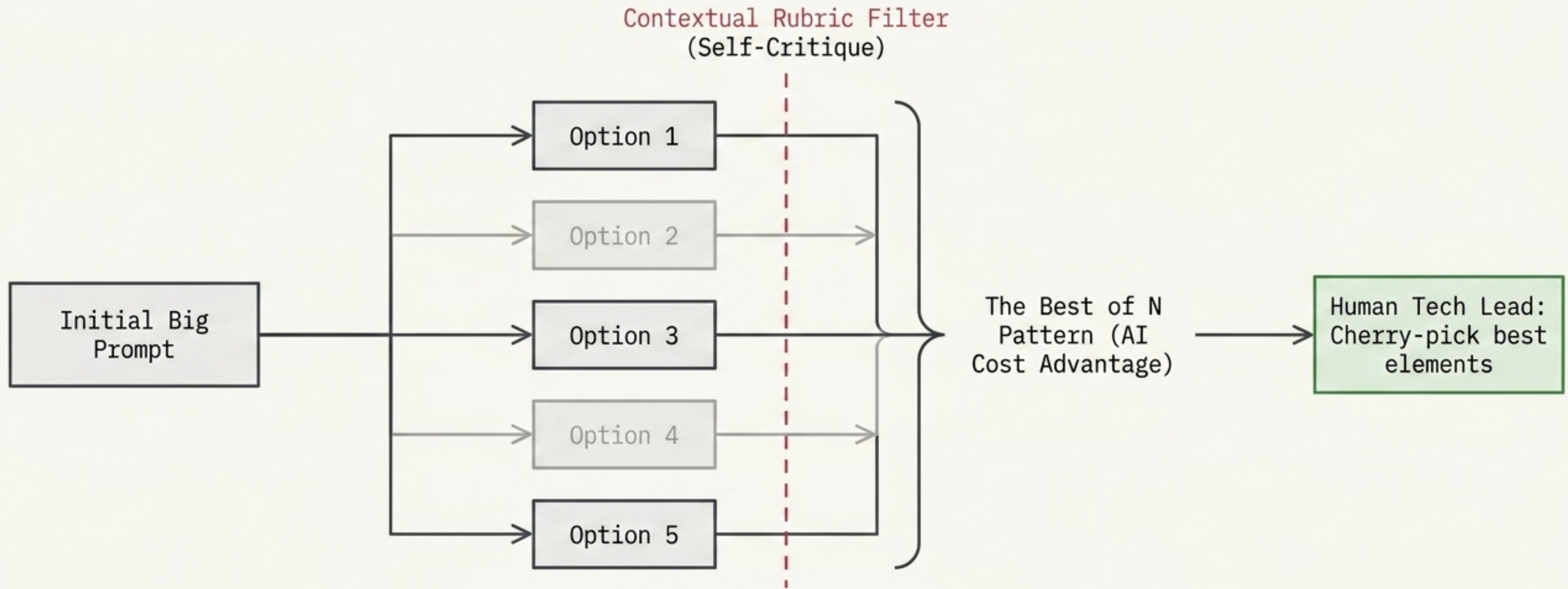
The AI-First Engineering Matrix

	Micromanaging Code (Traditional)	Orchestrating AI Labor (AI-First)
Role	Typist Writing boilerplate manually	Director Delegating tasks via big prompts
Prototyping	Single Path Committing to one architecture early	Best-of-N Generating 3-5 concurrent options and cherry-picking
Workflow	Sequential Linear feature building	Parallel Deploying AI subagents into concurrent git worktrees
Quality Assurance	Manual Review Human finds bugs post-development	Contextual Rubrics AI critiques its own code before presenting it

Chat, Craft, Scale: The workflow for system design

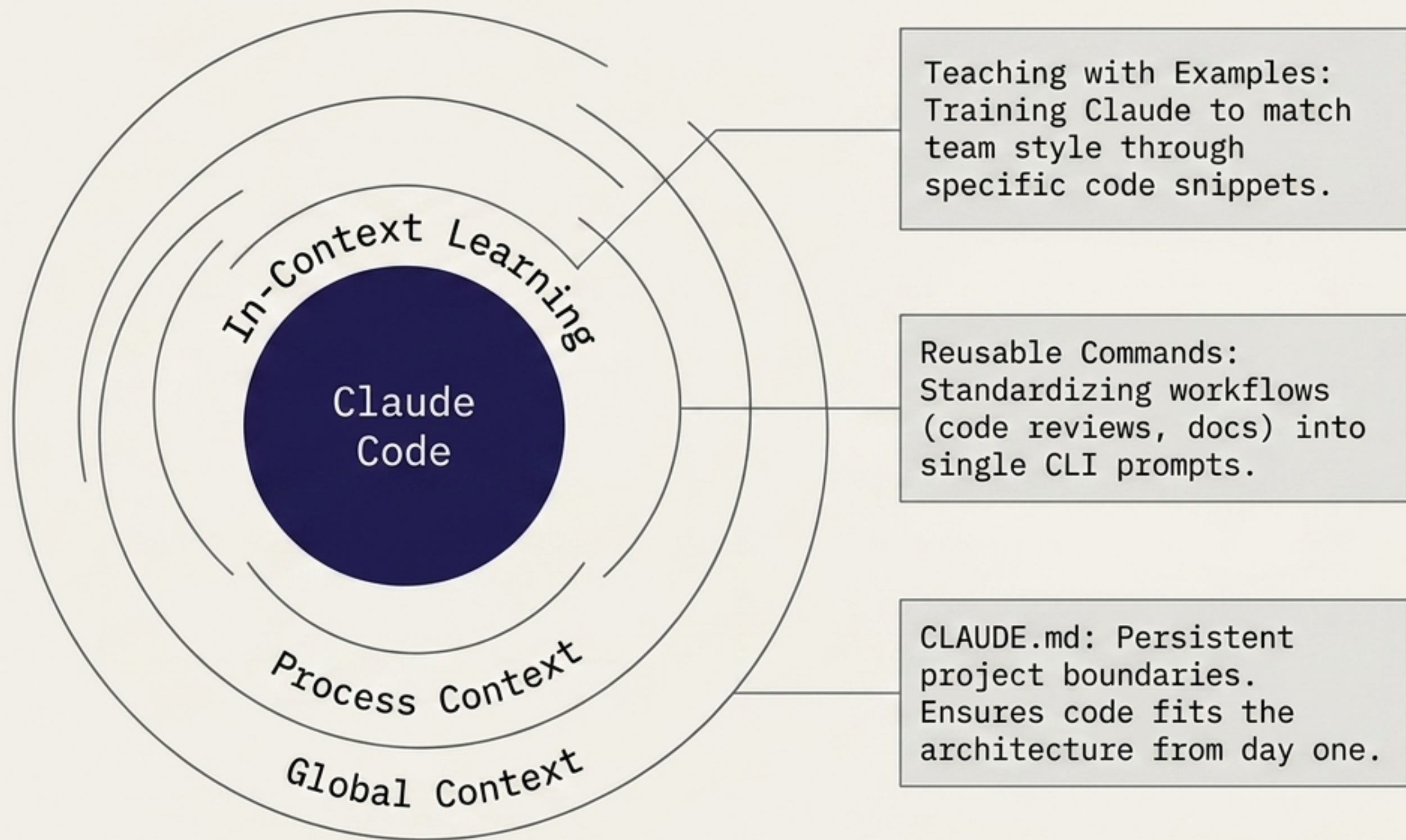


Leverage AI labor economics to explore multiple architectures simultaneously

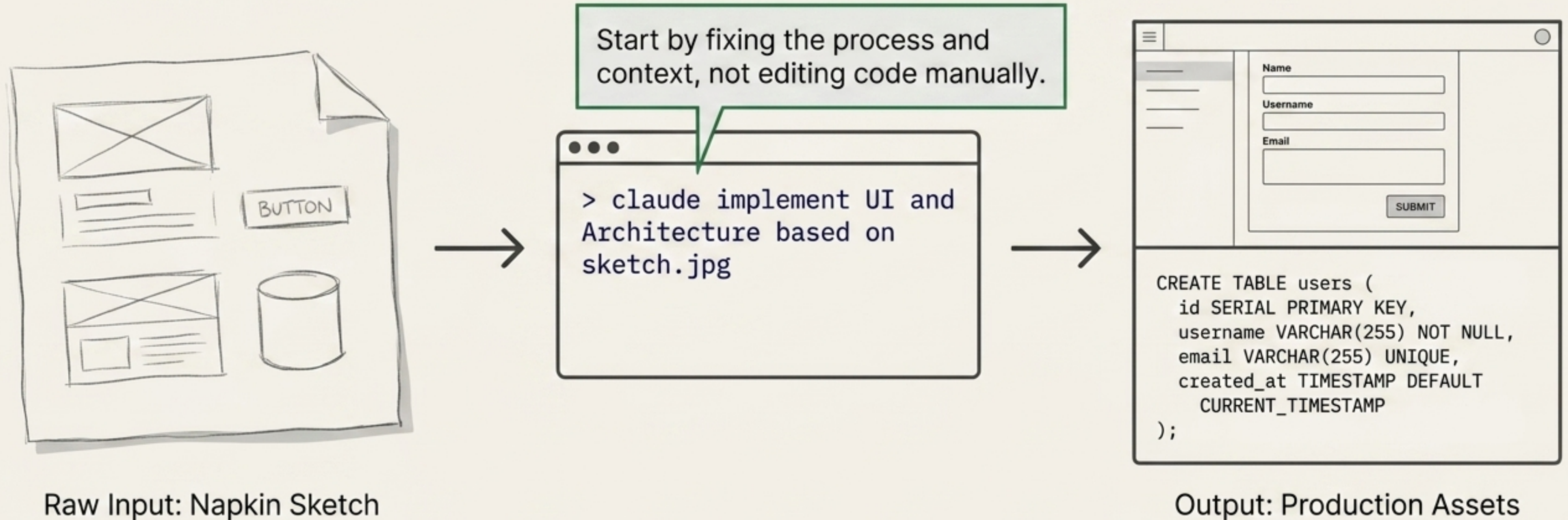


Key Takeaway: Never aim for a single perfect solution on the first try.
Generate multiple versions to identify the strongest approach.

Build autopilot onboarding with the Context Engine model

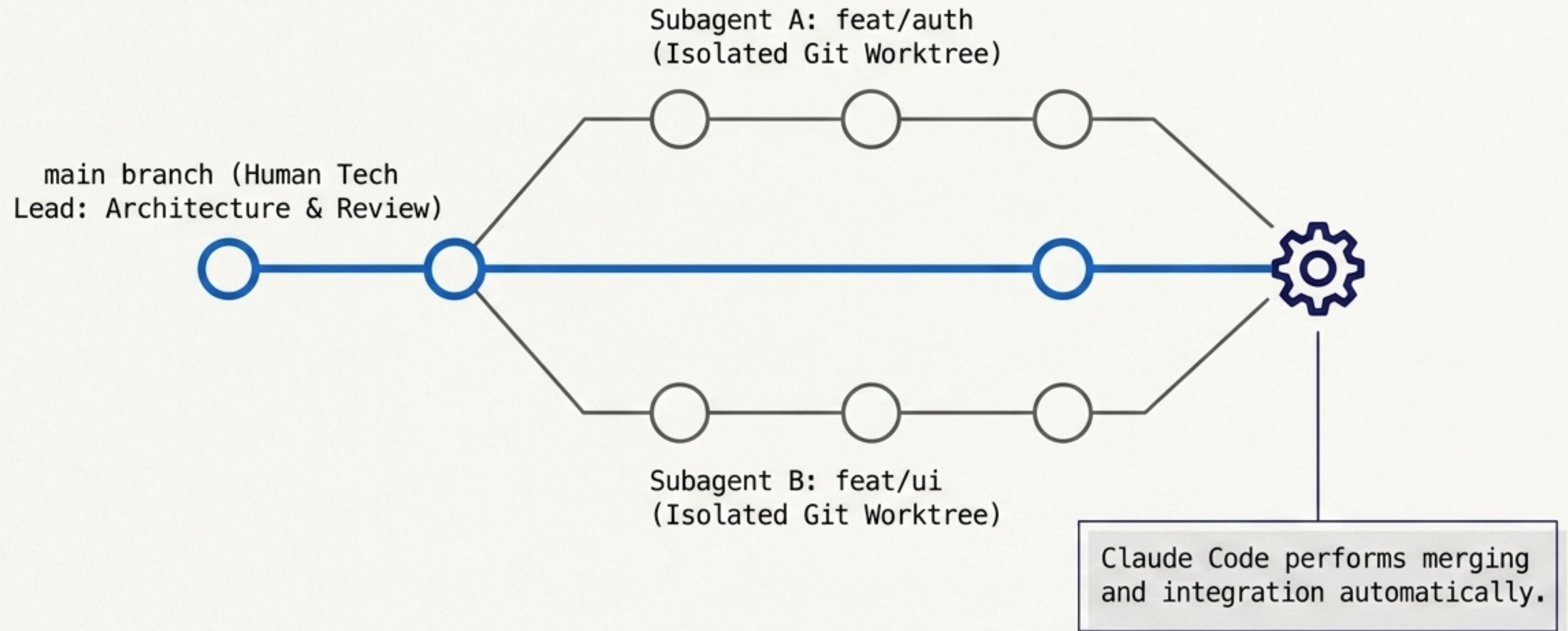


Multimodal prompting turns cocktail napkin sketches into production code

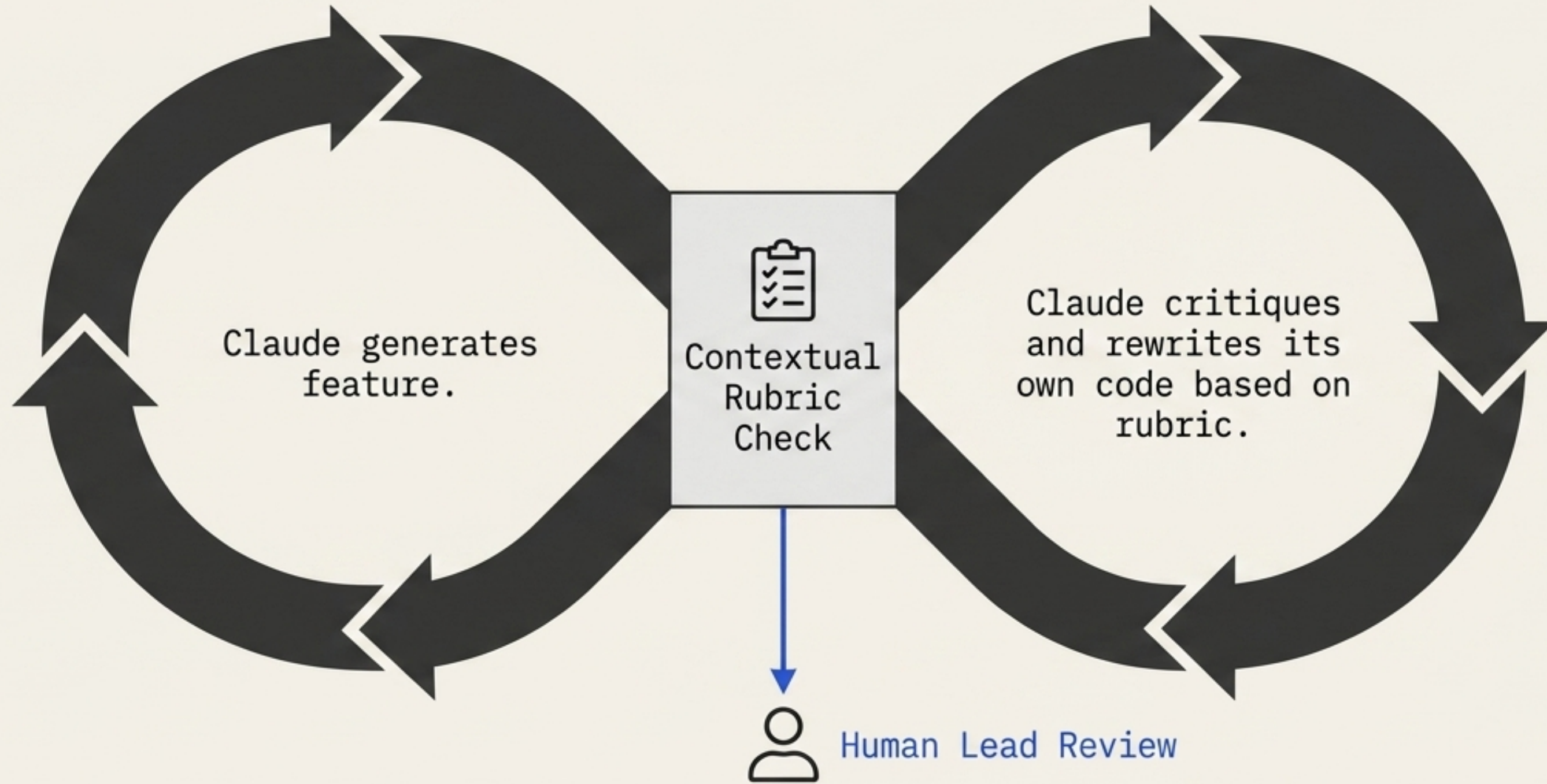


Cut feature development time from days to minutes.

Orchestrate parallel feature development via Git Worktrees



Catch bugs before they reach the human with contextual self-critique



Insight: You are Claude Code's hands, eyes, and ears. Ensure it checks its own work against strict rubrics before you ever look at a pull request.

Codebase architecture dictates AI token efficiency



Token Limit Exceeded



Monolithic files destroy AI context windows and degrade reasoning.

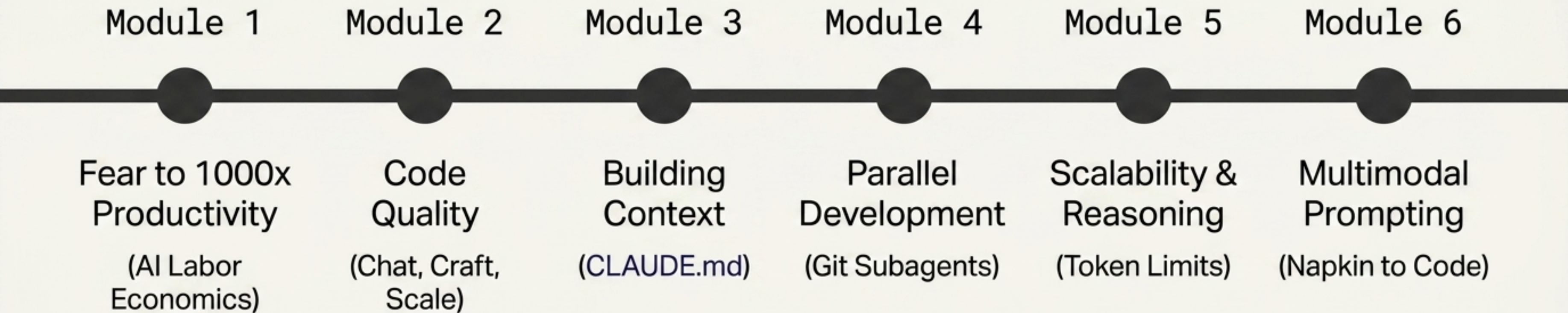


Optimal Context



Highly modular design, strict naming conventions, and logical structures maximize Claude Code's reasoning capabilities.

The Curriculum: 5 hours to rewire your engineering methodology



Acquired technical capabilities and toolsets

capabilities.json

[Software Architecture] [SDLC]

[Multimodal Prompts]

[Quality Assurance]

[Prompt Patterns] [Automation]

[Generative AI Agents]

[Token Optimization]

dependencies.lock



Anthropic Claude Code



Git (Version Control System)



Model Context Protocol (MCP)



Typescript

Target Profile: Who should adopt this paradigm?

> checking prerequisites...

[x] Basic software development experience

[x] Familiarity with version control (Git workflows)

[x] Ready to embrace AI while maintaining strict engineering standards

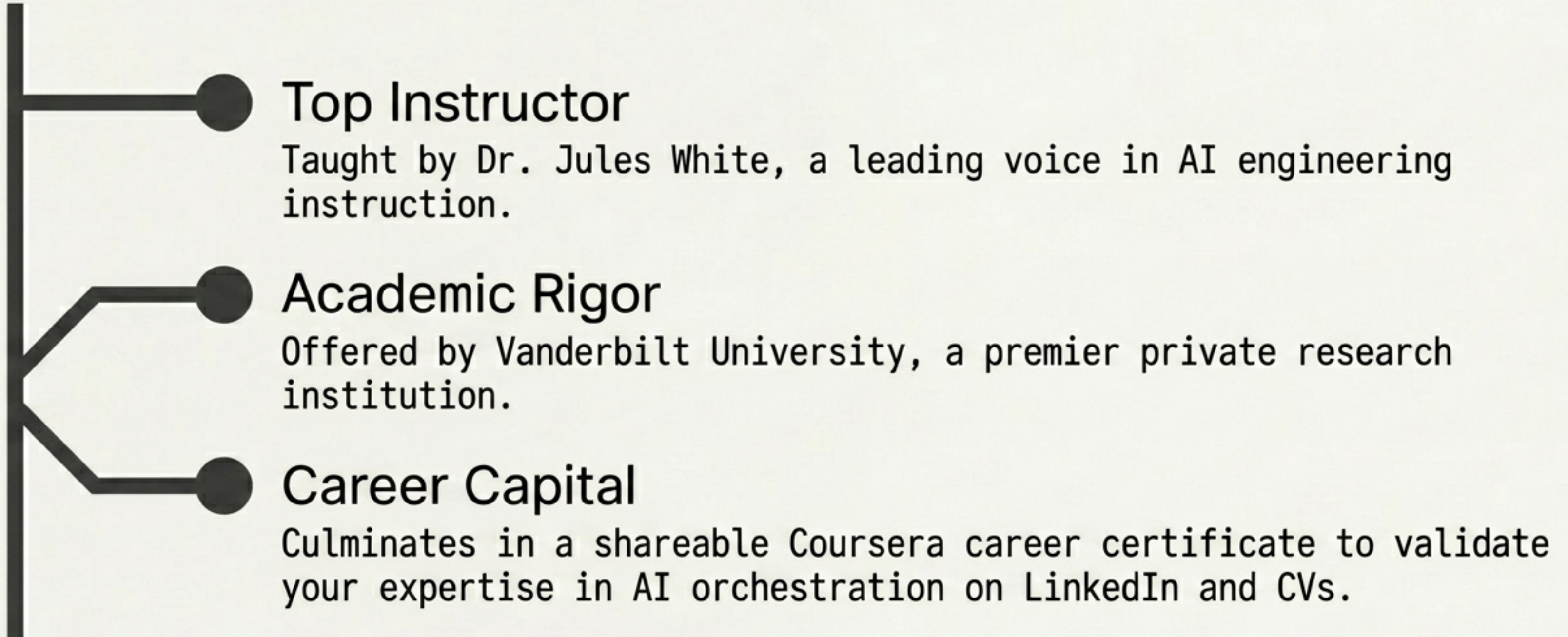
> executing match_roles.sh

Target Audience Identified: Software Engineers, Tech Leads, and Development Teams.

> generating promise.log

System Output: Never write boilerplate again. Build multiple prototypes before committing. Scale your personal productivity like you hired an entire team.

Institutional validation from Vanderbilt University



Start delegating. Stop micromanaging.

- > Requirements verified: Paid Claude Code subscription active.
- > Time investment: 5 flexible hours.
- > Action: Enroll in 'Claude Code: Software Engineering with Generative AI Agents'
- > Execute transformation? [Y/n] ■